CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

Clvex - Thetanuts v4

February 2026

Prepared by

ZanyBonzy

ArnieSec

Pelz

Introduction

A time-boxed security review of the **Clvex** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About **Clvex - Thetanuts v4**

Clvex is a subDAO and sub-protocol built on top of the Thetanuts v4 onchain options stack in this repository. The Clvex product surface is best thought of as:

1. A set of principal-protected "yield strategy vaults", the `YieldStrategyVault` family. These vaults only risk rebased yield, not user principal, when entering options strategies.
2. A thin periphery contract, `ClvexWrapper`, that makes these vaults usable with USDC on Base. It converts USDC ⇄ aBasUSDC through Aave and safely forwards any option positions to the user during exit.

The onchain building blocks that matter for Clvex are:

- Vault layer: `BaseVault` → `CashSettledVault` → `YieldStrategyVault` → strategy-specific vaults like `DirectionalStrategyVault` and `MeanRevertingCondorStrategyVault`.
- RFQ + option layer: `OptionFactory` + `OptionBook` + cash-settled option implementations.
- Periphery: `ClvexWrapper`.
- Oracles: Chainlink spot feeds + a Chainlink-derived TWAP consumer for cash settlement.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

***review commit hash* - 0419a447c2f33d3c048e8b70055d6c3437b4cf3b**

Scope

The following smart contracts were in scope of the audit:

Contract Path
src/vaults/BaseVault.sol
src/vaults/CashSettledVault.sol
src/vaults/YieldStrategyVault.sol
src/vaults/DirectionalStrategyVault.sol
src/vaults/MeanRevertingCondorStrategyVault.sol
src/vaults/utils/DateUtils.sol
src/vaults/utils/EventHandler.sol
src/vaults/utils/IBaseVault.sol
src/vaults/utils/VaultStorage.sol
src/vaults/utils/VaultUtils.sol
src/clvex/ClvexWrapper.sol
src/options/BaseOption.sol
src/options/CashSettledOption.sol
src/options/OptionFactory.sol
src/options/OptionBook.sol
src/options/CallSpreadOption.sol
src/options/PutSpreadOption.sol
src/options/IronCondorOption.sol
src/adapters/SkyRebasingAdapter.sol
src/adapters/ERC4626RebasingAdapter.sol
src/oracles/HistoricalPriceConsumerV3.sol
src/oracles/HistoricalPriceConsumerV3_TWAP.sol
src/oracles/HistoricalPriceConsumerV3_FIXEDPRICE.sol

Contract Path
src/interfaces/IStrategyVaults.sol
src/interfaces/IVaults.sol

The following number of issues were found, categorized by their severity:

- Critical & High: 7 issues
- Medium: 6 issues
- Low: 17 issues
- Informational: 5 issues

Findings Summary

ID	Title	Severity	Status
[C-01]	Referral fee unit mismatch enables cross collateral factory drain	Critical	Fixed
[C-02]	Double fee read in swapAndCall settlement leaks protocol fees and breaks fee accounting	Critical	Fixed
[C-03]	Zero contract RFQ can freeze cash settled yield vault withdrawals	Critical	Fixed
[C-04]	`ERC4626RebasingAdapter` double-conversion bug in `redeem` and `withdraw` causes fund loss and permanent DoS	Critical	Fixed
[H-01]	Split target mismatch can revert withdrawals	High	Fixed
[H-02]	First deposit inflation attack on `ERC4626RebasingAdapter` via vault share donation	High	Fixed
[H-03]	`ERC4626RebasingAdapter.withdraw` allowance bypass — spender can withdraw `2x` approved amount	High	Fixed
[M-01]	Requesters have no Incentive to Refer	Medium	Acknowledged
[M-02]	Limit mode RFQ can be force cancelled through withdraw before settlement	Medium	Fixed
[M-03]	Persistent zero share pending deposits can fill queue and block new deposits	Medium	Acknowledged
[M-04]	Quote-collateral valuation assumes hard $1 peg, causing mispricing during stablecoin depeg	Medium	Acknowledged
[M-05]	Push-based refunds in `_returnAssetsToCurrentWinner` can DoS entire RFQ lifecycle if token transfer fails	Medium	Fixed

ID	Title	Severity	Status
[M-06]	`OptionBook.swapAndFillOrder` reverts when DEX router does not consume exact source token amount	Medium	Acknowledged
[L-01]	`withdrawToUSDC` sends claimed `aBasUSDC` to users	Low	Acknowledged
[L-02]	Referrer Parameter Not Included in Signed Order	Low	Acknowledged
[L-03]	Authorized Routers Are Hardcoded and Cannot Be Updated	Low	Acknowledged
[L-04]	`getPriceAfterTimestamp` Uses Incorrect Comparison Logic	Low	Acknowledged
[L-05]	Deposit minimum checks use requested amount instead of received amount	Low	Acknowledged
[L-06]	ClvexWrapper `withdrawToUSDC` and `migrate` fail to capture non-compounding vault earnings, delivering mixed tokens and incomplete migrations	Low	Acknowledged
[L-07]	`ClvexWrapper.rescueETH` uses `transfer` which is deprecated	Low	Fixed
[L-08]	Insufficient Chainlink price feed staleness check — `updatedAt <= 0< code> is effectively a no-op`	Low	Fixed
[L-09]	Unchecked `delegatecall` return value in `BaseVault.handleOptionEvent` can silently leave stale option tracking	Low	Acknowledged
[L-10]	Stale role-transfer approvals resurrect when a previous holder regains the same role	Low	Fixed
[L-11]	Excess ETH sent during `BaseVault.withdraw` is not refunded to the user	Low	Acknowledged
[L-12]	AAVE_POOL should not be immutable	Low	Acknowledged
[L-13]	Malicious user can leave last depositor's tokens stuck	Low	Acknowledged
[L-14]	Assets Without Onchain Pricefeed Support	Low	Acknowledged
[L-15]	BaseVault auto settlement locks the revealed winner and skips post reveal price improvement	Low	Acknowledged
[L-16]	Rescue Address Not Updated When Factory Ownership Changes	Low	Acknowledged
[L-17]	Insufficient Oracle Validation – Missing Staleness, Zero, and L2 Sequencer Checks	Low	Acknowledged
[I-01]	`_syncIndex` can be optimized	Informational	Acknowledged
[I-02]	Allow Owners To `transferFrom` Without Allowance	Informational	Acknowledged

ID	Title	Severity	Status
[I-03]	`maxDeposit` Returns `type(uint256).max` on Revert, Violating Standard	Informational	Acknowledged
[I-04]	`OptionBook.cancelOrder` allows duplicate cancellation events for already-cancelled orders	Informational	Fixed
[I-05]	Redundant `whenNotPaused` modifier on `BaseVault.deposit`	Informational	Fixed

Detailed Findings

[C-01] Referral fee unit mismatch enables cross collateral factory drain

Severity

Impact: High

Likelihood: High

Description

The referral validation runs on caller supplied params, but `requestForQuotation` internal flow can later overwrite collateral fields from an existing option. This breaks the assumption that one referral id is tied to one collateral denomination.

```
function _validateReferralParams(QuotationParameters memory params,
uint256 referralId) internal view {
    if (referralId == 0 || referralId >= referralRegistry.length) revert
InvalidReferralId();
    QuotationParameters storage registeredParams =
referralRegistry[referralId];

    bool mismatch = false;

    if (
        params.collateral != registeredParams.collateral
            || params.collateralPriceFeed !=
registeredParams.collateralPriceFeed
            || params.implementation != registeredParams.implementation
            || params.expiryTimestamp != registeredParams.expiryTimestamp
            || params.isRequestingLongPosition !=
registeredParams.isRequestingLongPosition
            || keccak256(params.extraOptionData) !=
keccak256(registeredParams.extraOptionData)
    ) mismatch = true;
```

```
// Use existing option parameters
newParams.implementation = getOptionImplementation(existingOption);
newParams.collateral = getOptionCollateralToken(existingOption);
newParams.collateralPriceFeed = getOptionPriceFeed(existingOption);
newParams.numContracts = getOptionNumContracts(existingOption);
newParams.expiryTimestamp = getOptionExpiryTimestamp(existingOption);
newParams.collateralAmount = getOptionCollateralAmount(existingOption);
```

Referral balances are stored only by referral id, with no token key.

```
mapping(uint256 => uint256) public referralFees; // For referral-based
fees
```

Credits are added after settlement using the finalized quotation collateral, so balances become unit ambiguous.

```
address collateral = quotationParams[quotationId].collateral;

if (tracking.referralId != 0) {
    uint256 referrerFee = fee / 2;
    referralFees[tracking.referralId] += referrerFee;
    pendingFees[collateral] += fee - referrerFee;
}
```

Debits happen up front in `requestForQuotation` using `reservePrice` with no transfer from referral owner, and that value is later used to fund premium payout from factory token reserves.

```
uint256 balance = referralFees[tracking.referralId];
if (reservePrice > balance) revert InsufficientReferralBalance();

referralFees[tracking.referralId] = balance - reservePrice;
```

```
if (msg.sender == referralOwner[tracking.referralId]) {
    // requesterDeposit is covered by the referral balance, no collateral
transfer required
    emit ReferralRFQExecuted(tracking.referralId, quotationId,
requesterDeposit);
}
```

```
if (params.isRequestingLongPosition) {
    IERC20(params.collateral).safeTransfer(winner, winningPremium - fee);
}
```

An attacker can mix referral accrual in one token context/decimals and spend in another token context/decimals after parameter overwrite, which can drain real balances from the factory.

Recommendations

1. Disallow `existingOptionAddress` whenever referralId is non zero.
2. Revalidate referral constraints after final quotation params are resolved.
3. Make `referralFees` token scoped, for example referral id plus collateral token.
4. Enforce exact collateral match between referral registry entry and finalized quotation before referral debit.
5. Add tests for cross collateral referral accounting with mixed decimal collateral tokens.

Client

Fixed.

[C-02] Double fee read in swapAndCall settlement leaks protocol fees and breaks fee accounting

Severity

Impact: High

Likelihood: High

Description

The swapAndCall settlement path reads quote-collateral fee data twice through collateralPriceFeed, then uses the second value for refund math even though settlement accounting was already committed using the first value.

The first read happens during settlement. That value is written into feeCollected and credited into pendingFees and referralFees.

```
function _settleQuotation(uint256 quotationId, address winner, uint256
winningPremium) internal {
    QuotationParameters storage params = quotationParams[quotationId];
    QuotationState storage state = quotationStates[quotationId];

    // Calculate the fee based on the winning premium
    uint256 fee = _calculateFeeFromQuotationId(quotationId,
winningPremium);
```

```
    ...
    _finalizeQuotationState(quotationId, finalOptionContract, winner,
winningPremium, fee);
    _handlePremiumTransfer(quotationId, winner, winningPremium, fee);
    _distributeFees(quotationId, fee);
    ...
}
```

The fee source is external and read from collateralPriceFeed for quote collateral:

```
function _calculateFeeFromQuotationId(uint256 quotationId, uint256
winningPremium) internal view returns (uint256) {
    QuotationParameters storage params = quotationParams[quotationId];

    (bool isQuoteCollateral,,,) =
BaseOption(params.implementation).unpackOptionType();
    uint256 calculateFeePrice = 1e8;
    if (isQuoteCollateral) {
        (, int256 price,, uint256 updatedAt,) =
getAggregatorLatestRoundData(params.collateralPriceFeed);
        if (price <= 0) revert PriceMustBePositive();
        if (updatedAt <= 0) revert PriceFeedStale();
        if (getAggregatorDecimals(params.collateralPriceFeed) != 8) revert
InvalidPriceFeedDecimals();
        calculateFeePrice = uint256(price);
    }
    return calculateFee(params.numContracts, winningPremium,
calculateFeePrice);
}
```

After settlement already finished, routeSelfCall computes fee again and uses it to derive expected post-call balance:

```
// Calculate fees that remain in factory (common to both paths)
uint256 factoryFee = _calculateFeeFromQuotationId(quotationId,
targetAmount);

amountDelta = int256(factoryFee) - int256(initialRequesterDeposit);
```

swapAndCall then refunds any excess over that expected balance:

```
if (balanceDstAfterCall > uint256(balanceDstAfterAdjustment)) {
    uint256 excess = balanceDstAfterCall -
uint256(balanceDstAfterAdjustment);
    IERC20(swapDstToken).safeTransfer(msg.sender, excess);
}
```

If a malicious or non-deterministic collateralPriceFeed returns a lower second value, the caller receives the difference as a refund, while feeCollected, pendingFees, and referralFees still reflect the higher first value. Repeating this creates cumulative shortfall for that collateral token.

This can lead to:

1. pendingFees and referralFees becoming undercollateralized
2. withdrawFees reverting on insufficient token balance
3. settlement or cancellation paths failing when transfers rely on balances implied by inflated accounting
4. referral over-credit that can be converted into extraction of real pooled funds in later flows

Recommendations

1. Compute fee once per settlement and reuse that single value end to end. In swapAndCall, read the settled fee from quotation state after settlement instead of recalculating from collateralPriceFeed.
2. Remove post-settlement fee recomputation from routeSelfCall. amountDelta should be derived from the committed settlement fee, not a second oracle read.
3. Restrict accepted collateralPriceFeed addresses to approved deterministic feeds, or snapshot fee price once and bind it for the transaction path.
4. Add an invariant check for fee solvency per collateral token and a regression test with a feed that returns different prices across two reads in the same transaction.

Client

Fixed.

[C-03] Zero contract RFQ can freeze cash settled yield vault withdrawals

Severity

Impact: High

Likelihood: High

Description

The root issue starts in CashSettledVault. When the pending queue is full, the MIN_VALUE check is bypassed, so RFQ creation can continue even when RFQ budget is too small. For YieldStrategyVault children, RFQ budget can be zero, and contract sizing can return zero.

```
function determineRfqParams()
    public
    view
    virtual
```

```
    override
    returns (uint256 numContracts, uint256[] memory strikes, uint256)
{
    // assetIndex = 0;
    uint256 rfqAmount = calculateRfqAmount();

    if (
        calculateValueInUSD(rfqAmount, 0) < config.MIN_VALUE &&
        config.MAX_QUEUE != pendingDeposits.length
    ) {
        revert InsufficientCollateralForNewOption();
    }

    (numContracts, strikes) = calculateNumContracts(rfqAmount);

    return (numContracts, strikes, 0);
}
```

BaseVault then pushes this RFQ without rejecting zero contracts or zero collateral.

```
OptionFactory.QuotationParameters memory params = OptionFactory
    .QuotationParameters({
        requester: address(this),
        existingOptionAddress: address(0),
        collateral: address(collateralToken),
        collateralPriceFeed: address(assets[assetIndex].priceFeed),
        implementation: getOptionImplementation(assetIndex),
        strikes: strikes,
        numContracts: numContracts,
        requesterDeposit: 0,
        collateralAmount: 0,
        expiryTimestamp: getNextExpiryTimestamp(),
        offerEndTimestamp: block.timestamp + config.RFQ_DURATION,
        isRequestingLongPosition: isLongOptionStrategy(),
        convertToLimitOrder: true,
        extraOptionData: packExtraOptionData(assetIndex)
    });
params.collateralAmount = _BaseOption_calculateRequiredCollateral(
    params.implementation,
    strikes,
    numContracts
);
```

OptionFactory accepts a reveal that matches reserve, sets winner, and can transfer zero amount when RFQ size is zero.

```
} else if (state.currentWinner == address(0) && offerAmount ==
state.currentBestPriceOrReserve) {
    isCompetitive = true;
```

```
}
...
_handleBestOfferAcceptance(quotationId, offeror, offerAmount);
```

```
state.currentWinner = offeror;
state.currentBestPriceOrReserve = offerAmount;

uint256 collateralAmount = params.collateralAmount;
bool isRequestingLongPosition = params.isRequestingLongPosition;

IERC20(params.collateral)
    .safeTransferFrom(offeror, address(this), isRequestingLongPosition ?
collateralAmount : offerAmount);
```

Settlement then fails during option initialization because BaseOption rejects zero contracts and zero collateral.

```
require(numContracts > 0, "Num contracts must be greater than 0");
require(collateralAmount > 0, "Collateral amount must be greater than 0");
```

At this point the vault is stuck in a bad RFQ state:

- active RFQ remains active with non zero winner
- vault is not in limit mode since limit mode requires winner to be zero
- withdraw and pending withdraw are blocked
- each new handling attempt retries settlement and reverts again

```
function _requireWithdrawalAllowed() internal view {
    if (isRfqActive()) {
        if (!isRfqInLimitMode()) revert WithdrawalLockedDuringActiveRFQ();
    }
}
```

```
if (
    state.currentWinner != address(0) &&
    state.optionContract == address(0)
) {
    factory.settleQuotation(rfqId);
}
```

This is a protocol level denial of service for affected vault instances. All users cannot withdraw funds and vault is permanently DOSed. The attacker basically can pull off this attack for nearly 0 cost.

Recommendations

Add a hard guard in CashSettledVault and BaseVault RFQ creation path to reject numContracts equal to zero and collateralAmount equal to zero

Client

Fixed.

[C-04] `ERC4626RebasingAdapter` double-conversion bug in `redeem` and `withdraw` causes fund loss and permanent DoS

Severity

Impact: High

Likelihood: High

Description

`ERC4626RebasingAdapter.redeem` and `withdraw` compute `assetsAmount` and `scaledToBurn` using opposite conversion directions on the same `sharesAmount` input:

```
// redeem:
assetsAmount = Math.mulDiv(sharesAmount, _index, SCALING_FACTOR,
Math.Rounding.Down);   // treats input as SCALED
scaledToBurn = Math.mulDiv(sharesAmount, SCALING_FACTOR, _index,
Math.Rounding.Up);     // treats input as REBASING

// withdraw:
sharesAmount = Math.mulDiv(assetsAmount, SCALING_FACTOR, _index,
Math.Rounding.Up);     // assets → scaled
scaledToBurn = Math.mulDiv(sharesAmount, SCALING_FACTOR, _index,
Math.Rounding.Up);     // applies SF/index AGAIN
```

Both formulas cannot be correct for the same input because scaled and rebasing are different units (`rebasing = scaled * index / SF`). When `_index > SCALING_FACTOR` (i.e., any time yield has accrued):

- `redeem`: `assetsAmount` is inflated (over-requests assets from vault) while `scaledToBurn` is deflated (under-burns from owner)
- `withdraw`: `scaledToBurn = assets * (SF/index)²` instead of `assets * SF/index`, burning half the correct amount

Concrete example (`_index = 2e27`, Alice has 1000 scaled = 2000 rebasing = 2000 assets):

`redeem(maxRedeem)` **always reverts:**

- `maxRedeem(alice) = balanceOf(alice) = 2000` (rebasing)
- `assetsAmount = 2000 * 2e27 / 1e27 = 4000` — requests 4000 assets
- `scaledToBurn = 2000 * 1e27 / 2e27 = 1000` — balance check passes
- `vault.withdraw(4000, ...)` → **REVERTS** — only 2000 assets exist

`redeem(scaledBalance)` **under-burns:**

- `redeem(1000)`: `assetsAmount = 2000` (correct), `scaledToBurn = 500` (should be 1000)
- Alice gets all 2000 assets but only burns 500 scaled. 500 phantom shares remain backed by zero assets.
- Next `_syncIndex`: `currentAssets = 0`, `expectedAssets = 1000`, `_index → 0`. **Contract permanently bricked.**

No valid input exists that correctly redeems a full position.

Recommendations

Choose a single unit for `sharesAmount` and apply consistent conversions. If shares are **scaled** (matching internal storage):

```
// redeem: input is scaled shares
assetsAmount = Math.mulDiv(sharesAmount, _index, SCALING_FACTOR,
Math.Rounding.Down);
scaledToBurn = sharesAmount; // already scaled, no conversion needed

// withdraw: input is assets
scaledToBurn = Math.mulDiv(assetsAmount, SCALING_FACTOR, _index,
Math.Rounding.Up);
sharesAmount = scaledToBurn; // same unit
```

Client

Fixed.

[H-01] Split target mismatch can revert withdrawals

Severity

Impact: Medium

Likelihood: High

Description

The withdrawal split branch computes a split amount from the largest option position, then applies that amount to the lowest generation option. Those can be different options with different collateral sizes.

```
uint256 largestIndex = 0;
uint256 largestAmount = options[0].amount;
uint256 lowestGenerationIndex = 0;
uint256 lowestGeneration = options[0].splitGeneration;

for (uint256 i = 1; i < options.length; i++) {
    if (options[i].amount > largestAmount) {
        largestIndex = i;
        largestAmount = options[i].amount;
    }
    if (options[i].splitGeneration < lowestGeneration) {
        lowestGenerationIndex = i;
        lowestGeneration = options[i].splitGeneration;
    }
}
```

```
BaseOption optionToSplit = BaseOption(
    options[lowestGenerationIndex].optionContract
);

uint256 proposedSplitCollateral = (remainingShares *
    largestAmount) / optionShares;
```

If the chosen low generation option holds less collateral than the computed split amount, split reverts.

```
require(splitCollateralAmount > 0 && splitCollateralAmount <
collateralAmount, "Invalid split amount");
```

The current early returns only handle zero and too small split values. They do not verify that the aligned split amount is strictly below the selected option collateral. That leaves a DOS in withdraw for affected states. In bad distributions, exits can stay blocked until option state changes.

State that enables this is normal after prior splits because tracked options can diverge in both amount and split generation.

```
activeOptions[trackingInfo.strategy][trackingInfo.index].amount =
remainingAmount;
activeOptions[trackingInfo.strategy][trackingInfo.index].splitGeneration =
splitGeneration;

activeOptions[trackingInfo.strategy].push(
    OptionPosition({
        optionContract: newOption, amount: splitAmount, assetIndex:
assetIndex, splitGeneration: splitGeneration
```

```
    })
);
```

Recommendations

If the lowest generation option is too small, fall back to another option that meets the amount constraint, then prefer lowest generation among valid candidates.

Client

Fixed.

[H-02] First deposit inflation attack on `ERC4626RebasingAdapter` via vault share donation

Severity

Impact: High

Likelihood: Medium

Description

`ERC4626RebasingAdapter` maintains its own scaled-share accounting (`_scaledBalances`, `_scaledTotalSupply`, `_index`) on top of an external ERC-4626 vault. The adapter's `totalAssets()` is derived from the vault shares it holds:

```
function totalAssets() public view override returns (uint256) {
    return vault.convertToAssets(vault.balanceOf(address(this)));
}
```

Since vault shares are standard ERC-20 tokens, anyone can transfer them directly to the adapter, inflating `totalAssets()` without minting any scaled shares. Combined with the lack of a `scaledToMint > 0` check in `deposit` and `mint`, this enables a classic inflation attack:

Attack example (USDC):

1. Attacker calls `deposit(1, attacker)` — gets 1 scaled share. Adapter holds vault shares worth 1 wei.
2. Attacker deposits 1000 USDC directly into the underlying vault, receiving vault shares, then transfers those vault shares to the adapter contract. Cost: 1000 USDC.
3. On the next interaction, `_syncIndex()` runs:
    - `currentAssets ≈ 1000e6`, `expectedAssets = 1`
    - `_index` inflates from `1e27` to `~1e36`
4. Victim deposits 500 USDC:

    - `scaledToMint = FLOOR(500e6 * 1e27 / 1e36) = FLOOR(0.5) = 0`
    - Victim receives **0 shares**. Their 500 USDC enters the vault but is credited to no one.
5. Attacker redeems their 1 scaled share — `_syncIndex` incorporates the victim's deposit, and the attacker recovers 1000 + 500 = 1500 USDC. **Net profit: 500 USDC.**

The donation amount D sets the threshold: any single deposit < D receives 0 shares. Multiple victims can be accumulated before the attacker redeems. The external vault provides no protection because the attack targets the adapter's own index-based accounting, not the vault's internals.

Three factors make this exploitable:

1. **No dead shares** — no initial locked deposit to establish a minimum share price.
2. **No `require(scaledToMint > 0)`** — zero-share mints silently succeed.
3. **Freely transferable vault shares** — anyone can inflate `totalAssets()` via direct ERC-20 transfer.

Recommendations

Implement one or more of the following mitigations:

1. **Require non-zero shares on deposit/mint:**

```
require(scaledToMint > 0, "Deposit too small");
```

2. **Virtual offset (OpenZeppelin approach):** Add a virtual asset/share offset to prevent the index from being manipulable with small amounts:

```
function totalAssets() public view override returns (uint256) {
    return vault.convertToAssets(vault.balanceOf(address(this))) + 1;
}
```

Client

Fixed.

[H-03] `ERC4626RebasingAdapter.withdraw` allowance bypass — spender can withdraw `2x` approved amount

Severity

Impact: High

Likelihood: High

Description

`withdraw` deducts `sharesAmount` from the owner's allowance, where `sharesAmount` is in **scaled** terms. But allowances are set and consumed in **rebasing** terms by `transferFrom`. When `_index = 2 * SF`, scaled values are half the rebasing equivalent, so the allowance is consumed at half the expected rate:

```
// withdraw:
sharesAmount = Math.mulDiv(assetsAmount, SCALING_FACTOR, _index,
Math.Rounding.Up);  // ≈ assets/2 (scaled)
if (msg.sender != owner) {
    require(sharesAmount <= allowed, "Withdraw exceeds allowance");
    _allowances[owner][msg.sender] = allowed - sharesAmount;  // deducts
scaled from rebasing allowance
}
```

Concrete exploit (`_index = 2e27`, Alice has 1000 scaled = 2000 rebasing, approves Bob for 1000):

1. Bob calls `withdraw(1000 assets, bob, alice)`:
    - `sharesAmount = 500`, deducts 500 → allowance remaining: 500
    - `scaledToBurn = 250` (double-conversion from C-04), burns 250 from Alice
    - Bob receives 1000 assets
2. Bob calls `withdraw(1000 assets, bob, alice)` again:
    - `sharesAmount = 500`, deducts 500 → allowance remaining: 0
    - `scaledToBurn = 250`, burns 250 from Alice
    - Bob receives 1000 assets

Result: Bob withdrew **2000 assets** with only **1000 allowance**. Alice's scaled balance decreased by 500 (worth 1000 assets), but 2000 assets left the vault. The 1000 asset deficit is socialized across all remaining pool participants via index deflation on the next `_syncIndex`.

The multiplier scales with the index: when `_index = N * SF`, a spender can withdraw `N×` their approved amount.

Recommendations

Convert `sharesAmount` to rebasing terms before checking against the allowance, or store allowances in scaled terms consistently:

```
// Option A: convert to rebasing for allowance check
uint256 rebasingAmount = Math.mulDiv(scaledToBurn, _index, SCALING_FACTOR,
Math.Rounding.Up);
if (msg.sender != owner) {
    require(rebasingAmount <= allowed, "Withdraw exceeds allowance");
    _allowances[owner][msg.sender] = allowed - rebasingAmount;
}
```

Client

Fixed.

[M-01] Requesters have no Incentive to Refer

Severity

Impact: High

Likelihood: Medium

Description

When excess tokens are returned after settlement, the logic in `_handleExcessDeposit` gives the full excess to the requester if no referralId is used. For vaults acting as requesters, this creates a strong disincentive to use referrals: the excess is locked into `referralFees[referralId]` (conviction-based, only usable for identical trade parameters), while skipping the referral returns the excess directly to the vault for immediate reuse in the next cycle.

This defeats the purpose of the referral system for the primary users (vaults) and reduces overall referral adoption.

```
    function _handleExcessDeposit(uint256 quotationId, uint256
excessAmount) internal {
        if (excessAmount == 0) return;

        QuotationParameters storage params = quotationParams[quotationId];
        uint256 referralId = quotationTracking[quotationId].referralId;

        if (referralId != 0) {
            referralFees[referralId] += excessAmount;  // Added to
referrer's fee
        } else {
            IERC20(params.collateral).safeTransfer(params.requester,
excessAmount);  // Returned to requester
            emit CollateralReturned(quotationId, params.requester,
excessAmount);
        }
    }
```

Recommendations

Consider returning excess to the referrer (or splitting it) when a referralId is used, or provide an alternative incentive (e.g., boosted referral rewards) for vaults.

Client

Acknowledged.

[M-02] Limit mode RFQ can be force cancelled through withdraw before settlement

Severity

Impact: Medium

Likelihood: Medium

Description

The vault allows withdraw only when an active RFQ is in limit mode. In that same state, withdraw calls active RFQ handling, and active RFQ handling cancels the RFQ when there is no winner after reveal. This removes the RFQ immediately instead of leaving a short fill window for reserve settlement.

```
function isRfqInLimitMode() public view returns (bool) {
    if (activeRfqId == type(uint256).max) {
        return false;
    }
    (
        OptionFactory.QuotationParameters memory params,
        OptionFactory.QuotationState memory state
    ) = _OptionFactory_getQuotation(activeRfqId);
    return
        state.isActive &&
        params.convertToLimitOrder &&
        block.timestamp > params.offerEndTimestamp + REVEAL_WINDOW &&
        state.currentWinner == address(0);
}

function _handleActiveRfq() internal virtual {
    uint256 rfqId = activeRfqId;
    if (rfqId == type(uint256).max) {
        return; // No active RFQ
    }

    (
        OptionFactory.QuotationParameters memory params,
        OptionFactory.QuotationState memory state
    ) = _OptionFactory_getQuotation(rfqId);

    // If RFQ is already inactive, clear when cancelled; otherwise wait
for settlement callback/recovery
    if (!state.isActive) {
        if (state.optionContract == address(0)) {
            activeRfqId = type(uint256).max;
        }
```

```
        return;
    }

    // ONLY check timing and settle if needed
    if (block.timestamp > params.offerEndTimestamp + REVEAL_WINDOW) {
        if (
            state.currentWinner != address(0) &&
            state.optionContract == address(0)
        ) {
            factory.settleQuotation(rfqId);
        } else if (
            block.timestamp > params.offerEndTimestamp + REVEAL_WINDOW &&
            state.currentWinner == address(0)
        ) {
            _OptionFactory_cancelQuotation(rfqId);
            activeRfqId = type(uint256).max;
        }
    } else {
        revert RfqNotReadyForHandling();
    }
}
```

OptionFactory supports no winner reserve settlement in limit mode, but this path can be front run by any shareholder that calls withdraw first.

```
function settleQuotation(uint256 quotationId) external nonReentrant
validActiveQuotation(quotationId) {
    QuotationParameters storage params = quotationParams[quotationId];
    QuotationState storage state = quotationStates[quotationId];

    if (block.timestamp <= params.offerEndTimestamp + REVEAL_WINDOW)
revert RevealPeriodNotEnded();

    if (state.currentWinner != address(0)) {
        _settleQuotation(quotationId, state.currentWinner,
state.currentBestPriceOrReserve);
    } else if (params.convertToLimitOrder) {
        _handleBestOfferAcceptance(quotationId, msg.sender,
state.currentBestPriceOrReserve);
        _settleQuotation(quotationId, msg.sender,
state.currentBestPriceOrReserve);
    } else {
        revert NoWinnerToAccept();
    }
}
```

A dust shareholder can repeatedly cancel no winner RFQs at limit mode entry, causing cycle churn, missed fills, and reduced strategy execution reliability. This can also happen trough normal vault use.

Recommendations

Add a minimum post reveal grace period where no winner RFQs stay active for settlement attempts before vault side cancellation is allowed.

Client

Fixed.

[M-03] Persistent zero share pending deposits can fill queue and block new deposits

Severity

Impact: Medium

Likelihood: Medium

Description

The issue comes from keeping zero share pending deposits in the queue for YieldStrategyVault.

```
function _shouldKeepZeroShareDeposit()
    internal
    view
    override
    returns (bool)
{
    return true;
}
```

When pending deposits are processed, a deposit can end with sharesToMint as zero while effectiveDeposit is still positive. In YieldStrategyVault, that entry is kept instead of removed.

```
uint256 sharesToMint = 0;
if (totalAssetsUSD > totalPendingDepositValue) {
    sharesToMint =
        (calculateValueInUSD(effectiveDeposit, assetIndex) *
            cachedTotalSupply) /
        (totalAssetsUSD - totalPendingDepositValue);
}

if (sharesToMint > 0) {
    _mint(
        pendingDeposit.user,
        sharesToMint,
        effectiveDeposit,
        assetIndex
    );
```

```
        unchecked {
            processedCount++;
        }
    } else if (effectiveDeposit > 0) {
        emit ZeroShareDeposit(
            pendingDeposit.user,
            pendingDeposit.amount,
            effectiveDeposit,
            assetIndex
        );
        if (_shouldKeepZeroShareDeposit()) {
            pendingDeposits[writeIndex] = pendingDeposit;
            unchecked {
                writeIndex++;
            }
        } else {
            unchecked {
                processedCount++;
            }
        }
    }
```

Only the owner of a pending deposit can remove it through withdrawPendingDeposits. Other users cannot clear attacker entries.

```
    for (uint256 i = 0; i < pendingDeposits.length; i++) {
        if (
            pendingDeposits[i].user == msg.sender &&
            pendingDeposits[i].amount > 0
        ) {
            amounts[pendingDeposits[i].assetIndex] += pendingDeposits[i]
                .amount;
```

The queue has a hard cap. Once full, all new deposits revert.

```
    if (pendingDeposits.length >= config.MAX_QUEUE)
        revert DepositQueueFull();
```

This allows a griefing pattern where a malicious actor accumulates retained zero share entries and refuses to call withdrawPendingDeposits, causing persistent queue saturation and blocking fresh deposits indefinitely for almost no cost.

Recommendations

Consider not retaining zero share deposits in the main pendingDeposits queue

Client

Acknowledged.

[M-04] Quote-collateral valuation assumes hard $1 peg, causing mispricing during stablecoin depeg

Severity

Impact: Medium

Likelihood: Medium

Description

`YieldStrategyVault.calculateValueInUSD()` values quote-like assets using a fixed conversion path when `isUnderlyingBaseAsset == false`:

- `USD value = amount * PRICE_DECIMALS / 10^decimals`

For `DirectionalStrategyVault` quote-collateral configurations (e.g., `aBasUSDC`), this implies a hard assumption that 1 token equals $1 at all times, rather than using a live oracle for the quote asset.

This creates a valuation mismatch whenever the quote asset depegs:

- If the quote asset trades below $1 (e.g., $0.90), the vault overestimates collateral value in USD terms.
- If it trades above $1 (e.g., $1.05), the vault underestimates collateral value in USD terms.

Because USD conversion is used by strategy sizing and RFQ/value guardrails, depeg events can distort risk allocation and contract sizing:

- At $0.90, the system may size positions as if it has more real USD collateral than it actually does.
- At $1.05, the system becomes overly conservative and underutilizes capital.

Recommendations

1. Use a live quote-asset USD oracle path for quote-collateral valuation instead of a fixed $1 assumption.
2. Add depeg guardrails (e.g., pause or scale down risk) when quote price deviates beyond a configured band (such as +/-1-2%).

Client

Acknowledged.

[M-05] Push-based refunds in `_returnAssetsToCurrentWinner` can DoS entire RFQ lifecycle if token transfer fails

Severity

Impact: High

Likelihood: Low

Description

`OptionFactory._returnAssetsToCurrentWinner` uses a push pattern (`safeTransfer`) to refund the current winning bidder. If this transfer reverts for any reason, every operation that depends on it is permanently bricked, including new bids, settlement, cancellation, and all vault operations gated behind `_handleActiveRfq()`.

```
// OptionFactory.sol:772-790
function _returnAssetsToCurrentWinner(uint256 quotationId) internal {
    QuotationParameters storage params = quotationParams[quotationId];
    QuotationState storage state = quotationStates[quotationId];

    address currentWinner = state.currentWinner;
    if (currentWinner != address(0)) {
        uint256 amountToReturn;
        if (params.isRequestingLongPosition) {
            amountToReturn = params.collateralAmount;
        } else {
            amountToReturn = state.currentBestPriceOrReserve;
        }
        state.currentWinner = address(0);
        IERC20(params.collateral).safeTransfer(currentWinner,
amountToReturn); // <-- reverts if transfer fails
        emit CollateralReturned(quotationId, currentWinner,
amountToReturn);
    }
}
```

This function is called across four critical paths:

Call site	Context
`_revealOffer` (line 1031)	Outbid → refund previous winner
`cancelQuotation` (line 1670)	RFQ cancellation → refund winner
`settleQuotationEarlyByOrderBook` (line 1345)	OptionBook settlement → refund winner
`_refundWinnerAndCancel` (line 801)	State-change abort → refund winner

If `safeTransfer` reverts (e.g., USDC/USDT blacklists the winner's address, or the collateral token is paused), all four paths revert. The RFQ enters a permanently stuck state:

- **No new bids accepted** — `_revealOffer` reverts, locking in the stuck winner
- **No cancellation** — `cancelQuotation` reverts, requester cannot recover their deposit

- **No settlement** — for long vaults, `_handlePremiumTransfer` also transfers to the winner (line 850), compounding the issue

Recommendations

1. **Wrap the refund in a try/catch and escrow failed refunds** for the winner to claim later via a pull pattern:

```
mapping(address => mapping(address => uint256)) public claimableRefunds;
// user => token => amount

function _returnAssetsToCurrentWinner(uint256 quotationId) internal {
    QuotationParameters storage params = quotationParams[quotationId];
    QuotationState storage state = quotationStates[quotationId];

    address currentWinner = state.currentWinner;
    if (currentWinner != address(0)) {
        uint256 amountToReturn;
        if (params.isRequestingLongPosition) {
            amountToReturn = params.collateralAmount;
        } else {
            amountToReturn = state.currentBestPriceOrReserve;
        }
        state.currentWinner = address(0);

        try IERC20(params.collateral).transfer(currentWinner,
amountToReturn) returns (bool success) {
            if (!success) {
                claimableRefunds[currentWinner][params.collateral] +=
amountToReturn;
            } else {
                emit CollateralReturned(quotationId, currentWinner,
amountToReturn);
            }
        } catch {
            claimableRefunds[currentWinner][params.collateral] +=
amountToReturn;
        }
    }
}

function claimRefund(address token) external nonReentrant {
    uint256 amount = claimableRefunds[msg.sender][token];
    if (amount == 0) revert NothingToClaim();
    claimableRefunds[msg.sender][token] = 0;
    IERC20(token).safeTransfer(msg.sender, amount);
}
```

Client

Fixed.

[M-06] `OptionBook.swapAndFillOrder` reverts when DEX router does not consume exact source token amount

Severity

Impact: Medium

Likelihood: High

Description

`OptionBook.swapAndFillOrder` enforces a strict equality check on the source token balance after the router swap:

```
// OptionBook.sol:298-299
uint256 balanceAfter = IERC20(swapSrcToken).balanceOf(address(this));
require(balanceAfter == balanceBefore, "Source token balance mismatch");
```

This requires the router to consume **exactly** `swapSrcAmount` — not one wei more, not one wei less. If any source tokens remain unconsumed, `balanceAfter > balanceBefore` and the entire transaction reverts.

DEX aggregation routers (1inch v6, Paraswap, 0x, Odos) routinely do not consume the full approved amount:

- **Partial routing**: Aggregators split swaps across multiple pools. Rounding across pools can leave dust unconsumed.
- **Exact output swaps**: When the user specifies a minimum output rather than an exact input, the router consumes only what's needed and returns the rest.
- **Dynamic path optimization**: Between transaction submission and execution, optimal routing may shift, resulting in slightly less input being consumed.

The `OptionFactory.swapAndCall` handles this correctly by returning any excess source tokens to the caller:

```
// OptionFactory.sol:1863-1867
uint256 balanceSrcAfter = getBalance(swapSrcToken);
if (balanceSrcAfter > balanceSrcBefore) {
    IERC20(swapSrcToken).safeTransfer(msg.sender, balanceSrcAfter -
balanceSrcBefore);
}
```

The factory even acknowledges this behavior in its own comments (line 1860): *"swapRouter sometimes don't use full approval amount."* Yet the OptionBook's `swapAndFillOrder` does not apply the same pattern, making it functionally unreliable with the same set of authorized routers.

Since the transaction reverts atomically, no funds are lost. However, `swapAndFillOrder` becomes unusable in practice for any swap route that produces even 1 wei of leftover input — effectively breaking the swap-and-fill feature for most real-world usage with aggregation routers.

Recommendations

Return excess source tokens to the caller instead of reverting, matching the factory's pattern:

```
uint256 balanceAfter = IERC20(swapSrcToken).balanceOf(address(this));
if (balanceAfter > balanceBefore) {
    IERC20(swapSrcToken).safeTransfer(msg.sender, balanceAfter -
balanceBefore);
}
// Remove the strict equality require
```

Client

Acknowledged.

[L-01] `withdrawToUSDC` sends claimed `aBasUSDC` to users

Description

During ClvexWrapper's interactions with YieldStrategyVault, `aBasUSDC` earnings are claimed on behalf of the caller and sent to him. In cases when users deal with USDC directly, this might inconvenient for them. For instance in `withdrawToUSDC`, `vault.withdraw` claims `aBasUSDC` for the users, despite the user intending to withdraw all USDC (especially for users who only deal with USDC).

```
        // 3. Withdraw from vault
        uint256 preABasUSDCBalance = aBasUSDC.balanceOf(address(this));
        try vault.withdraw{value: msg.value}(shares) returns (uint256[]
memory) {} // Burns wrapper's shares → _claimEarnings sends aBasUSDC to
users.

        ...
        usdcAmount = AAVE_POOL.withdraw(..., msg.sender);  // User gets
only USDC
```

Recommendations

Consider forwarding claimed aBasUSDC to wrapper and converting all to USDC for the callers.

Client

Acknowledged.

[L-02] Referrer Parameter Not Included in Signed Order

Description

The `fillOrder` function in `OptionBook.sol` allows anyone to execute a signed limit order by providing the order, the maker's signature, and a `referrer` address:

```
function fillOrder(Order calldata order, bytes calldata signature, address
referrer)
    public
    nonReentrant
    returns (address optionAddress)
{
    return _fillOrderInternal(order, signature, referrer);
}
```

The `Order` struct contains all trade parameters (maker, collateral, strikes, etc.) and is signed by the maker. However, the `referrer` address is not included in the signed hash. This allows a malicious user to calling `fillOrder` with the same order and signature before a legitimate user but substituting a different address as `referrer` especially if they are the `referrer`. As a result, stealing/griefing attacks can be performed against other referrers/the factory.

For example,

1. Legitimate user might have `referrer` A in mind, while the malicious user passes in `referrer`, redirecting the funds. This steals the fee if the caller.
2. A referrer with higher `referrerFeeSplitBps` may be passed in, reducing allocations to the factory.

```
        uint256 referrerFee;
        {
            referrerFee = 0;
            // Note: Factory-originated trades bypass referrer fees
entirely.
            // This is intentional - factory uses its own _distributeFees
for fee splitting.
            // Referrers never earn on factory-mediated settlements, even
if referrer address is passed.
            if (msg.sender != address(factory) && referrer != address(0))
{
```

```
                uint256 feeSplit = referrerFeeSplitBps[referrer];
                if (feeSplit > 0) {
                    referrerFee = (fee * feeSplit) / 10000;
                    fees[order.collateral][referrer] += referrerFee;
                    emit ReferralFeePaid(referrer, order.collateral,
referrerFee, optionAddress);
                }
            }
            // Accrue protocol fee (total fee minus referrer fee)
            // Skip when factory is taker - factory handles its own fee
accounting via _distributeFees
            // This prevents double-accounting: OptionBook would track
fees in its ledger while
            // factory has already credited them to
pendingFees/referralFees. By skipping this,
            // fees stay where factory's accounting expects them.
            if (msg.sender != address(factory)) {
                fees[order.collateral][address(factory)] += fee -
referrerFee;
            }
        }
```

PS: In these cases, the cost of premium or collateral will be borne by caller, hence the lower severity as the attack may be too expensive to be profitable for the caller.

Recommendations

Include `referrer` in `LIMIT_ORDER_TYPEHASH` and `hashOrder` so the maker signs the intended referrer.

Client

Acknowledged.

[L-03] Authorized Routers Are Hardcoded and Cannot Be Updated

Description

In the `OptionFactory` constructor, several router addresses are hardcoded and marked as authorized:

```
        // Initialize authorizedRouters
        authorizedRouters[0x6131B5fae19EA4f9D964eAc0408E4408b66337b5] =
true; // Kyber
        authorizedRouters[0x111111125421cA6dc452d289314280a0f8842A65] =
true; // 1inch v6
        authorizedRouters[0xDef1C0ded9bec7F1a1670819833240f027b25EfF] =
true; // 0x Exchange Proxy
        authorizedRouters[0x19cEeAd7105607Cd444F5ad10dd51356436095a1] =
```

```
true; // Odos Router V2
        authorizedRouters[0x6A000F20005980200259B80c5102003040001068] =
true; // Paraswap AugustusV6
```

There is no function to update this list after deployment. If any router is sunset, upgraded to a new address, or becomes malicious, the owner cannot revoke its authorization or add a new router without redeploying the factory. This reduces the system's flexibility and may force reliance on outdated or insecure routers.

Recommendations

Add an owner-only function to set the authorized status of any router address.

Client

Acknowledged.

[L-04] `getPriceAfterTimestamp` Uses Incorrect Comparison Logic

Description

The function `getPriceAfterTimestamp` in the oracle contract is intended to return the price recorded after a given timestamp. The current implementation returns the price only if `timeStamp >= priceTime`:

```
function getPriceAfterTimestamp(address, uint256 timeStamp) public view
returns (int256) {
    if (timeStamp >= priceTime) return priceX1e6;
    revert("Block not found");
}
```

However, to obtain a price that is valid after a specific timestamp, we need the recorded price time to be **at or after** that timestamp, i.e., `priceTime >= timeStamp`. The current condition is reversed: it only returns the price when the requested time is after the recorded price time, which effectively means it returns a price that was set before the requested time, i.e., it returns a stale price. Conversely, if a price was set after the requested time (which is the correct one), the function reverts.

Example:

- Price set at `t=100`.
- User queries price after `t=50`. Expected: price at `t=100` (valid). Actual: `50 >= 100` is false → revert.
- User queries price after `t=150`. Expected: revert (no price after 150). Actual: `150 >= 100` is true → returns stale price set at 100.

Recommendations

Reverse the condition to correctly check that the recorded price time is after the requested time:

Client

Acknowledged.

[L-05] Deposit minimum checks use requested amount instead of received amount

Description

BaseVault depositOnBehalf computes depositValueInUSD from the user provided amount before transfer, then later computes actualReceived from balance delta. The minimum value gates still use depositValueInUSD, while minting and queued accounting use actualReceived.

```
uint256 depositValueInUSD = calculateValueInUSD(amount, assetIndex);

// Measure balance before transfer to track actual received amount
// Tokens come from msg.sender (could be wrapper), shares go to
beneficiary
uint256 balanceBefore = _getAssetBalance(assetIndex);
assets[assetIndex].token.safeTransferFrom(
    msg.sender,
    address(this),
    amount
);
uint256 balanceAfter = _getAssetBalance(assetIndex);

// Calculate actual received amount (handles rebasing token precision)
uint256 actualReceived = balanceAfter - balanceBefore;

if (totalSupply() == 0) {
    if (depositValueInUSD < config.MIN_VALUE)
        revert FirstDepositBelowMinValue();
    uint256 initialShares = 1e18;
    initialMint = true;
    _mint(beneficiary, initialShares, actualReceived, assetIndex);
    lastRecordedValuePerShare = _calculateValuePerShare();
    emit ValuePerShareUpdated(lastRecordedValuePerShare);
} else {
    uint256 minDepositValue = config.MIN_VALUE / config.MAX_QUEUE;
    if (depositValueInUSD < minDepositValue)
        revert DepositValueTooLow();
```

This creates a mismatch between validation and accounting:

- First deposit can pass MIN_VALUE using nominal amount while actual received value is lower
- Subsequent deposits can pass queue minimum checks while actual queued value is lower
- Deposit quality guards become weaker whenever transfer behavior causes received amount to be lower than requested amount

YieldStrategyVault uses the same BaseVault path through super call.

```
function depositOnBehalf(
    address beneficiary,
    uint256 amount,
    uint256 assetIndex
) public override {
    if (isInRecoveryMode) revert CannotDepositInRecoveryMode();
    _claimEarnings(beneficiary);
    super.depositOnBehalf(beneficiary, amount, assetIndex);
}
```

Recommendations

depositValueInUSD should be calculated from actualReceived

Client

Acknowledged.

[L-06] ClvexWrapper `withdrawToUSDC` and `migrate` fail to capture non-compounding vault earnings, delivering mixed tokens and incomplete migrations

Description

When users withdraw or migrate from a **non-compounding** `YieldStrategyVault` (e.g., `DirectionalStrategyVault`) through `ClvexWrapper`, their accumulated earnings bypass the wrapper's conversion/migration logic and are sent directly to the user as raw `aBasUSDC`.

The root cause is the `_beforeTokenTransfer` hook in `YieldStrategyVault`:

```
// YieldStrategyVault.sol:312-324
function _beforeTokenTransfer(address from, address to, uint256 amount)
internal virtual override {
    super._beforeTokenTransfer(from, to, amount);
    if (from != address(0) && to != address(0) && !isCompounding()) {
        _claimEarnings(from);  // <-- sends aBasUSDC directly to `from`
(the user)
        _claimEarnings(to);
    }
}
```

When the wrapper calls `safeTransferFrom(user, wrapper, shares)`, this hook fires *before* the share transfer and sends all pending earnings as aBasUSDC to the user via `_claimEarnings(user)` → `safeTransfer(user, pending)`.

The wrapper then snapshots its own aBasUSDC balance *after* this hook has already diverted the earnings:

```
// ClvexWrapper.sol:318
uint256 preABasUSDCBalance = aBasUSDC.balanceOf(address(this));
```

Since the earnings went to the user (not the wrapper), the `aBasUSDCReceived` delta at line 328/381 only reflects the proportional vault balance from `_executeWithdrawal`, excluding earnings entirely.

Impact on `withdrawToUSDC`:

The function name and interface promise USDC output. Instead, the user receives:

- **USDC** — from the Aave conversion of their proportional vault balance (returned in `usdcAmount`)
- **aBasUSDC** — earnings sent directly to the user's address, unconverted

The `usdcAmount` return value and the `WithdrawToUSDC` event understate the actual value withdrawn. Any smart contract integrating with `withdrawToUSDC` that relies on the return value will have an incomplete accounting of the withdrawal. If the caller is a contract that doesn't handle aBasUSDC, those earnings tokens become stuck.

Impact on `migrate`:

The function is intended to atomically move a user's full position from vault A to vault B. Instead:

- Only the proportional vault balance (`aBasUSDCMigrated`) is deposited into the destination vault
- Earnings leak out as aBasUSDC to the user's wallet, excluded from `depositOnBehalf`

The `aBasUSDCMigrated` return value and `Migration` event understate the value extracted from the source vault. The user's position in the destination vault is smaller than their actual value in the source vault. Users may not notice the leaked aBasUSDC, resulting in idle funds that stop earning options yield.

Recommendations

1. **Snapshot the user's aBasUSDC balance before the `safeTransferFrom`** to capture any earnings paid out during the hook, then include those in the conversion/migration:

```
// withdrawToUSDC fix
uint256 preUserABasUSDC = aBasUSDC.balanceOf(msg.sender);
IERC20(address(vault)).safeTransferFrom(msg.sender, address(this),
shares);
uint256 earningsToUser = aBasUSDC.balanceOf(msg.sender) - preUserABasUSDC;

// After vault.withdraw, pull earnings back from user and include in Aave
conversion
if (earningsToUser > 0) {
```

```
    aBasUSDC.safeTransferFrom(msg.sender, address(this), earningsToUser);
    aBasUSDCReceived += earningsToUser; // include in Aave withdrawal
}
```

2. **For `migrate`**, apply the same approach — pull leaked earnings back and include them in `depositOnBehalf` to the destination vault.

Client

Acknowledged.

[L-07] `ClvexWrapper.rescueETH` uses `transfer` which is deprecated

Description

`ClvexWrapper.rescueETH` uses Solidity's `.transfer()` to send ETH:

```
function rescueETH(address payable to) external onlyOwner {
    to.transfer(address(this).balance);
}
```

`.transfer()` forwards only 2300 gas, which is insufficient for smart contract recipients (e.g., multisigs, Gnosis Safes) whose `receive()`/`fallback()` functions exceed the gas stipend. Since `onlyOwner` is likely a multisig or governance contract, the rescue could fail when sending to itself or other contract wallets.

Recommendations

Replace `.transfer()` with a low-level `.call{value: ...}("")` and check the return value:

```
function rescueETH(address payable to) external onlyOwner {
    (bool success, ) = to.call{value: address(this).balance}("");
    require(success, "ETH transfer failed");
}
```

Client

Fixed.

[L-08] Insufficient Chainlink price feed staleness check — `updatedAt <= 0` is effectively a no-op

Description

Both `OptionFactory._calculateFeeFromQuotationId` and `OptionBook._createOption` validate the Chainlink price feed staleness with:

```
// OptionFactory.sol:1798
if (updatedAt <= 0) revert PriceFeedStale();

// OptionBook.sol:422
if (updatedAt <= 0) revert OptionFactory.PriceFeedStale();
```

`updatedAt` is a `uint256` timestamp returned by Chainlink's `latestRoundData()`. A `uint256` can never be negative, so the only value this check catches is `updatedAt == 0` — meaning the round was never initialized. It does **not** detect stale data.

A Chainlink price feed that last updated hours or even days ago would pass this check as long as `updatedAt > 0`. For example, if the feed's last update was at timestamp `1700000000` (November 2023) and the current time is February 2026, the check passes — the price is over two years stale but is still accepted for fee calculation.

This affects fee computation for quote-collateral options in both the RFQ settlement path (`OptionFactory._calculateFeeFromQuotationId`, line 1796) and the OptionBook order path (`OptionBook._createOption`, line 420). A stale price leads to incorrect fee amounts:

- **If stale price is higher than real price**: fees are overcharged, reducing the premium the recipient receives
- **If stale price is lower than real price**: fees are undercharged, the protocol collects less than intended

Recommendations

Check `updatedAt` against a maximum acceptable staleness threshold relative to `block.timestamp`:

```
uint256 MAX_STALENESS = 1 hours; // adjust based on the feed's expected
heartbeat
if (block.timestamp - updatedAt > MAX_STALENESS) revert PriceFeedStale();
```

Client

Fixed.

[L-09] Unchecked `delegatecall` return value in `BaseVault.handleOptionEvent` can silently leave

stale option tracking

Description

`BaseVault.handleOptionEvent` delegates event processing to `EventHandler` via `delegatecall` without checking the return value:

```
// BaseVault.sol:1478
EVENT_HANDLER.delegatecall(callData); // return value NOT checked
```

If `EventHandler` reverts (e.g., out-of-gas from the limited gas forwarded by `_notifyParty`, or an unexpected state edge case), the failure is silently ignored. The option's own operation (settlement, split, transfer) continues because `_notifyParty` is non-blocking, but the vault's `activeOptions` tracking arrays are never updated.

This leaves phantom option positions that inflate `getTotalAssets()` and can skew share price calculations until `_handleExpiredOptions()` cleans them up on the next cycle.

Recommendations

Check the `delegatecall` return value and emit a warning on failure so the issue is observable:

```
(bool success,) = EVENT_HANDLER.delegatecall(callData);
if (!success) emit EventHandlerFailed(optionAddress, eventType);
```

Client

Acknowledged.

[L-10] Stale role-transfer approvals resurrect when a previous holder regains the same role

Description

`BaseOption.transfer` only clears the specific approval used for the current transfer (`buyerAllowance[buyer][msg.sender] = false`). All other approvals previously granted by the outgoing role holder remain in storage:

```
if (isBuyer) {
    buyerAllowance[buyer][msg.sender] = false; // only clears THIS
caller's approval
    buyer = target;
```

```
} else {
    sellerAllowance[seller][msg.sender] = false;
    seller = target;
}
```

Because the auth check keys on the current role holder's address (`buyerAllowance[buyer][msg.sender]`), these dormant approvals automatically reactivate if the same address ever regains the role — which is an explicitly supported flow via `existingOptionAddress` resales (e.g., Alice sells buyer role to Bob, later buys it back).

Example:

1. Alice is the `buyer` and approves both Charlie and Dave via `approveTransfer`.
2. Alice's buyer role is transferred to Bob (e.g., via factory settlement). Only the transferring caller's approval is cleared — Charlie's and Dave's approvals persist in storage as `buyerAllowance[Alice][Charlie] = true` and `buyerAllowance[Alice][Dave] = true`.
3. These approvals are dormant while Bob holds the role (auth checks `buyerAllowance[Bob][...]`).
4. Alice later buys back the same option position via a resale RFQ.
5. Charlie and Dave can now transfer Alice's role without fresh consent, potentially stealing an in-the-money position.

Recommendations

Clear all approvals for the outgoing role holder on transfer, or use a nonce/generation counter that invalidates all prior approvals when the role changes hands:

```
// Option A: generation counter
mapping(address => uint256) public buyerApprovalNonce;

function transfer(bool isBuyer, address target) external virtual
nonReentrant {
    // ... auth checks ...
    if (isBuyer) {
        buyerApprovalNonce[buyer]++;  // invalidates all of buyer's
approvals
        buyer = target;
    } else {
        sellerApprovalNonce[seller]++;
        seller = target;
    }
}
```

Client

Fixed.

[L-11] Excess ETH sent during `BaseVault.withdraw` is not refunded to the user

Description

`BaseVault.withdraw` requires users to send ETH to cover option splitting fees. The function checks that sufficient ETH was sent but does not refund any excess:

```
function withdraw(uint256 shares) public payable virtual nonReentrant
returns (uint256[] memory amounts) {
    // ...
    amounts = _executeWithdrawal(shares);
    if (msg.value < amounts[activeOptionsStrategies]) {
        revert
InsufficientETHSentDuringWithdrawal(amounts[activeOptionsStrategies]);
    }
    _burn(msg.sender, shares);
    // ... no refund of msg.value - amounts[activeOptionsStrategies]
}
```

The exact ETH fee is only known after `_executeWithdrawal` completes (it depends on how many options need splitting), so the natspec instructs users to estimate via a static call with their "entire ETH balance." Users will routinely overpay, and the excess ETH remains in the vault contract, eventually swept by the owner via `withdrawETH()`.

Recommendations

Refund excess ETH to the caller after the fee check:

```
uint256 ethFee = amounts[activeOptionsStrategies];
if (msg.value < ethFee) {
    revert InsufficientETHSentDuringWithdrawal(ethFee);
}
uint256 refund = msg.value - ethFee;
if (refund > 0) {
    (bool success, ) = msg.sender.call{value: refund}("");
    require(success, "ETH refund failed");
}
```

Client

Acknowledged.

[L-12] AAVE_POOL should not be immutable

Description

In `ClvexWrapper`, `AAVE_POOL` is immutable and set once in the constructor. Aave pools can be migrated to new addresses as recommended in their docs. This will break aave dependent functonalities in the wrapper. Depositing and withdrawing will be permanently dossed. Additionally, max approval granted is permanent and irrevocable without a revoke function, risking misuse if the pool is compromised.

```
IAavePool public immutable AAVE_POOL;

constructor(...) {
    ...
    AAVE_POOL = IAavePool(_aavePool);

    // Pre-approve Aave pool for USDC supply
    USDC.approve(_aavePool, type(uint256).max);
}
```

See here: https://aave.com/docs/developers/smart-contracts/pool-addresses-provider

Recommendations

Use aave's `PoolAddressProvider` contract to query the current pool address instead for every interaction. Or use a setter function instead. In the function, approval should be revoked from the older pool, and granted to the newer pool.

Client

Acknowledged.

[L-13] Malicious user can leave last depositor's tokens stuck

Description

In `BaseVault.withdraw`, there is an explicit invariant check that reverts if `totalSupply() == 0 && pendingDeposits.length > 0`. This check relies on pending depositors voluntarily calling `withdrawPendingDeposits()` before the last shareholder exits. A malicious last depositor (or griefing via dust) can intentionally leave pending deposits in the queue, causing the final withdrawal to revert and locking the last user's funds.

```
    // INVARIANT: totalSupply==0 implies pendingDeposits.length==0
    if (totalSupply() == 0 && pendingDeposits.length > 0) {
        revert CannotWithdrawAllWithPendingDeposits();
    }
```

Recommendations

Automatically clear or refund remaining pending deposits when `totalSupply()` would reach zero.

Client

Acknowledged.

[L-14] Assets Without Onchain Pricefeed Support

Description

There's a comment in HistoricalPriceConsumerV3_FIXEDPRICE.sol that mentions the contract being used for assets without an onchain pricefeed. It can also be observed that HistoricalPriceConsumerV3_FIXEDPRICE.sol doesn't inherit HistoricalPriceConsumerV3Base or extend any of chainlink feed functions. Functions like `latestRoundData`, `decimals` etc are missing.

```
// For tokens without chainlink, this would act as temporary stand-in
until there's a onchain pricefeed
// Each asset type will own price set by a oracle
// All functions will ignore address provided in call, since price feed
address is not available yet

contract HistoricalPriceConsumerV3_FIXEDPRICE {
```

As was mentioned in issue L-17, certain parts of the codebase use different sources to query prices, sometimes chainlink's `latestRoundData` is queried, sometimes, `getLatestPrice` is used. This means for asset without onchain feeds, there'll be no way to calculate TWAP or get latestRoundData.

Recommendations

Extend the contract to introduce these functions, and consider standardizing the price query methods across the codebase.

Client

Acknowledged.

[L-15] BaseVault auto settlement locks the revealed winner and skips post reveal price improvement

Description

BaseVault handleActiveRfq settles immediately after reveal when a winner exists, and it calls OptionFactory settleQuotation directly.

```
if (block.timestamp > params.offerEndTimestamp + REVEAL_WINDOW) {
    if (
        state.currentWinner != address(0) &&
        state.optionContract == address(0)
    ) {
        factory.settleQuotation(rfqId);
    } else if (
        block.timestamp > params.offerEndTimestamp + REVEAL_WINDOW &&
        state.currentWinner == address(0)
    ) {
        _OptionFactory_cancelQuotation(rfqId);
        activeRfqId = type(uint256).max;
    }
} else {
    revert RfqNotReadyForHandling();
}
```

In OptionFactory settleQuotation, winner settlement takes priority. Limit order fallback is only used when there is no winner.

```
if (state.currentWinner != address(0)) {
    _settleQuotation(quotationId, state.currentWinner,
state.currentBestPriceOrReserve);
} else if (params.convertToLimitOrder) {
    _handleBestOfferAcceptance(quotationId, msg.sender,
state.currentBestPriceOrReserve);
    _settleQuotation(quotationId, msg.sender,
state.currentBestPriceOrReserve);
} else {
    revert NoWinnerToAccept();
}
```

OptionFactory does support replacing an existing winner with a better OptionBook price, but that path requires requester control and competitiveness checks.

```
if (params.isRequestingLongPosition) {
    if (hasWinner) {
        if (premiumAmount >= state.currentBestPriceOrReserve) revert
OrderNotCompetitive();
    } else {
        if (premiumAmount > params.requesterDeposit) revert
OrderNotCompetitive();
    }
} else {
    if (hasWinner) {
```

```
        if (premiumAmount <= state.currentBestPriceOrReserve) revert
OrderNotCompetitive();
    } else {
        if (premiumAmount < state.currentBestPriceOrReserve) revert
OrderNotCompetitive();
    }
}
```

BaseVault has no dedicated route to use that improvement flow, and executeExternalCall blocks calling the factory directly.

```
if (target == address(factory)) revert CallToInfrastructureNotAllowed();
```

This creates execution opportunity cost. A better post reveal fill can exist, but the BaseVault auto path settles the old revealed winner first, which can lower LP yield over time.

Recommendations

Consider allowing optionBook settlement if it is better than the winner.

Client

Acknowledged.

[L-16] Rescue Address Not Updated When Factory Ownership Changes

Description

When an option is created (via `_createOption` in `OptionBook` or `OptionFactory`, or via `split` in `BaseOption`), the `rescueAddress` is set to the factory owner at that moment. This address is stored immutably in the option contract and is never updated, even if the factory owner changes later.

```
        // Initialize option contract
        BaseOption(optionAddress)
            .initialize(
                BaseOption.OptionParams({
                    collateralToken: order.collateral,
                    chainlinkPriceFeed: order.priceFeed,
                    historicalTWAPConsumer:
address(factory.historicalTWAPConsumer()),
                    buyer: h.long,
                    seller: h.short,
                    strikes: order.strikes,
                    expiryTimestamp: order.expiry,
```

```
                    twapPeriod: factory.TWAP_PERIOD(),
                    numContracts: order.numContracts,
                    collateralAmount: collateralAmount,
                    rescueAddress: factory.owner(), //Owner address
                    factoryAddress: address(factory),
                    extraOptionData: order.extraOptionData
                })
            );
```

Later, if the factory owner is transferred to a new address, the old rescue address becomes a backdoor into options, especially in cases of suspected address compromise.

Recommendations

Introduce a mechanism to update the rescue address in options, perhaps by allowing the factory to call a function on each option to update the rescue address, or by storing a rescue address in the factory and having options read it dynamically via a call.

Client

Acknowledged.

[L-17] Insufficient Oracle Validation – Missing Staleness, Zero, and L2 Sequencer Checks

Description

PS: Oracle issues are being combined for concise purposes.

Multiple functions across `OptionBook`, `OptionFactory`, and the vault contracts fetch prices from the oracle contracts but lack comprehensive validation:

1. **Missing staleness checks** – Several locations only check `updatedAt > 0` without enforcing a maximum age. For example, in `OptionBook._createOption`:

   ```
   (, int256 price,, uint256 updatedAt,) =
   AggregatorProxy(order.priceFeed).latestRoundData();
   if (price <= 0) revert OptionFactory.PriceMustBePositive();
   if (updatedAt <= 0) revert OptionFactory.PriceFeedStale();
   ```

   A price could be hours or days old and still be accepted.

2. **Missing zero-price checks** – In some places, price is not validated to be >0 before use. For instance, `extractLatestRoundData` only checks `timeStamp > 0`:

```
        if (minPremiumAmount > 0) {
            uint256 premiumAmount = (order.price * order.numContracts) /
PRICE_DECIMALS;
            (bool isQuoteCollateral,,,) =
BaseOption(order.implementation).unpackOptionType();
            uint8 collateralDecimals =
IERC20Metadata(order.collateral).decimals();
            int256 premiumUsd8;
            if (isQuoteCollateral) {
                premiumUsd8 = int256(premiumAmount * 1e8 / (10 **
collateralDecimals));
            } else {
                int256 px =
HistoricalPriceConsumerV3_TWAP(factory.historicalTWAPConsumer())
                    .getLatestPrice(AggregatorProxy(order.priceFeed));
                premiumUsd8 = (int256(premiumAmount) * px) / int256(10 **
collateralDecimals);
            }
            require(premiumUsd8 >= int256(minPremiumAmount), "Premium <
minimum");
        }
    }
```

A zero price would pass this check and be used.

Another point to note is `getLatestPrice` in HistoricalPriceConsumerV3_FIXEDPRICE.sol may be queried before the oracle sets a price via `setPrice`, as a result, zero price will be returned and potentially used in such cases where price is not validated.

3. **Missing L2 sequencer checks** – On Base Chain and other L2 networks, the sequencer can be temporarily down; using prices during such periods can lead to stale data. The code does not query the sequencer uptime feed as recommended by chainlink docs.

4. **Hardcoded staleness threshold** – In `BaseVault._getValidatedPrice`, a global `MAX_ORACLE_STALENESS` (24 hours) is used. However, different assets may require different freshness thresholds. Using a single value for all assets may be too permissive or too strict.

5. **Inconsistent Source Oracle Contracts** - In certain cases, HistoricalPriceConsumerV3 contracts are used. In certain cases, chainlink feeds are queried directly. In `_createOption` for instance, OptionBook queries `latestRoundData` from chainlink feed. In OptionBook.sol, `_validateEconomics` uses `getLatestPrice` from HistoricalPriceConsumerV3 contract although the function calls `latestRoundData` under the hood.

Recommendations

Add staleness checks, prices, L2 sequencer checks and use asset specific staleness. Also, consider using a consistent source of token pricing across the codebase.

Client

Acknowledged.

[I-01] `_syncIndex` can be optimized

Description

In `ERC4626RebasingAdapter._syncIndex`, the index update logic uses separate `if`/`else if` branches for `currentAssets > expectedAssets` and `currentAssets < expectedAssets`. Both branches perform the exact same operation (`_index = Math.mulDiv(_index, currentAssets, expectedAssets, Math.Rounding.Down)`). When the values are equal, the index remains unchanged. This can be simplified into a single calculation, improving code clarity and saving gas.

Recommendations

Replace the `if`/`else if` with a single `mulDiv` operation. The equal case is naturally handled since the ratio becomes 1. Something like this works:

```
    function _syncIndex() internal returns (uint256) {
        uint256 currentAssets = totalAssets();
        uint256 expectedAssets = Math.mulDiv(_scaledTotalSupply, _index,
SCALING_FACTOR, Math.Rounding.Down);

        if (expectedAssets == 0) {
            return _index;
        }

        // Can be simplified to single mulDiv (works for >, <, and ==
cases)
        _index = Math.mulDiv(_index, currentAssets, expectedAssets,
Math.Rounding.Down);

        return _index;
    }
```

Client

Acknowledged.

[I-02] Allow Owners To `transferFrom` Without Allowance

Description

In `SkyRebasingAdapter.transferFrom` (and similar in ERC4626RebasingAdapter), the allowance check is performed unconditionally, even when `from == msg.sender` (the owner). This forces the owner

to set an explicit allowance to themselves first, which is unnecessary and might be inconvenient.

```
    function transferFromUnderlying(address from, address to, uint256
underlyingAmount)
        public
        returns (uint256 rebasedAmount)
    {
        require(from != address(0), "Transfer from zero address");
        require(to != address(0), "Transfer to zero address");

        rebasedAmount = _sharesToAssets(underlyingAmount);
        uint256 currentAllowance = super.allowance(from,
_msgSender());//Always checked even if caller is owner
        require(currentAllowance >= underlyingAmount, "Insufficient
allowance");

        if (currentAllowance != type(uint256).max) {
            super._approve(from, _msgSender(), currentAllowance -
underlyingAmount);
        }

        emit AssetConversion(underlyingAmount, rebasedAmount);
        super._transfer(from, to, underlyingAmount);
        return rebasedAmount;
    }
```

Recommendations

Consider skipping allowance check when `msg.sender` is `from`/`owner`

Client

Acknowledged.

[I-03] `maxDeposit` Returns `type(uint256).max` on Revert, Violating Standard

Description

According to ERC-4626, `maxDeposit` should not revert and should return the maximum amount that can be deposited, or `0` if deposit is not possible. In `ERC4626RebasingAdapter`, the implementation uses a try/catch and returns `type(uint256).max` if the underlying vault's `maxDeposit` reverts (which ideally, it should not do, according to EIP4626 standard). Returning `type(uint256).max` suggests an unlimited deposit capacity, which may be incorrect and could cause upstream contracts to attempt deposits that will fail. The correct behavior is to return `0` when the underlying vault's function reverts, indicating that deposits are temporarily disabled.

Recommendations

Change the catch block to return `0` instead of `type(uint256).max`:

Client

Acknowledged.

[I-04] `OptionBook.cancelOrder` allows duplicate cancellation events for already-cancelled orders

Description

`OptionBook._cancelOrder` writes the cancellation sentinel unconditionally:

```
// OptionBook.sol:588-593
function _cancelOrder(Order calldata order, address caller) internal {
    require(caller == order.maker, "Only maker can cancel");
    uint256 nonce = computeNonce(order);
    amountFilled[nonce] = type(uint256).max;
    emit OrderCancelled(nonce, caller);
}
```

If the same order (same nonce) is cancelled again, the function succeeds and emits another `OrderCancelled` event, even though the order was already cancelled.

Recommendations

Add an explicit guard to reject repeated cancellation attempts:

```
uint256 nonce = computeNonce(order);
require(amountFilled[nonce] != type(uint256).max, "Already cancelled");
amountFilled[nonce] = type(uint256).max;
emit OrderCancelled(nonce, caller);
```

Client

Fixed.

[I-05] Redundant `whenNotPaused` modifier on `BaseVault.deposit`

Description

`BaseVault.deposit` applies the `whenNotPaused` modifier, but the function body simply delegates to `depositOnBehalf`, which already has its own `whenNotPaused` modifier. The pause check is executed twice for every `deposit` call, wasting gas.

```
function deposit(uint256 amount, uint256 assetIndex) public virtual
whenNotPaused {
    depositOnBehalf(msg.sender, amount, assetIndex);
}
```

Recommendations

Remove `whenNotPaused` from `deposit` since `depositOnBehalf` already enforces it:

```
function deposit(uint256 amount, uint256 assetIndex) public virtual {
    depositOnBehalf(msg.sender, amount, assetIndex);
}
```

Client

Fixed.